



Vidyavardhini's College of Engineering and Technology

Department of Artificial Intelligence & Data Science

AY: 2026-27

Class:	TE-AI&DS	Semester:	V
Course Code:	2015111	Course Name:	Statistics for Machine Learning and Data Science Lab

Name of Student:	ASHLESHA .S. BHOIR
Roll No. :	03
Experiment No.:	6
Title of the Experiment:	Develop and Evaluate Regression Models using Statistical Performance Measures
Date of Performance:	19-8-2026
Date of Submission:	2-10-2026

Evaluation

Performance Indicator	Max. Marks	Marks Obtained
Performance	5	
Understanding	5	
Journal work and timely submission	10	
Total	20	

Performance Indicator	Exceed Expectations (EE)	Meet Expectations (ME)	Below Expectations (BE)
Performance	4-5	2-3	1
Understanding	4-5	2-3	1
Journal work and timely submission	8-10	5-8	1-4

Checked by

Name of Faculty : Dr. Tatwadarshi Nagarhalli

Signature :

Date :



Vidyavardhini's College of Engineering and Technology

Department of Artificial Intelligence & Data Science

Aim: To develop regression models for predicting a continuous variable using the Pima Indians Diabetes Dataset and evaluate their performance using appropriate statistical metrics.

Objectives

After completing this experiment, the student will be able to:

1. Develop regression models to predict a continuous health-related variable from medical attributes.
2. Evaluate and interpret the performance of regression models using appropriate statistical measures.

Tools Required

- Python 3.x
- Google Colab / Jupyter Notebook
- Pandas
- NumPy
- Matplotlib
- Seaborn
- Scikit-learn

Dataset Details

Dataset: Pima Indians Diabetes Dataset

The dataset contains medical information for **768 female patients**.

For this experiment, **BMI may be selected as the continuous target variable**, while other relevant medical attributes are used as input features.



Vidyavardhini's College of Engineering and Technology

Department of Artificial Intelligence & Data Science

Procedure

1. Load the Pima Indians Diabetes Dataset.
2. Select a continuous target variable and relevant predictor variables.
3. Check and preprocess missing or invalid values.
4. Divide the dataset into training and testing sets.
5. Develop a suitable regression model.
6. Train the model using the training data.
7. Generate predictions for the test data.
8. Evaluate the model using appropriate performance metrics.
9. Analyze residuals and compare actual and predicted values.
10. Interpret the model results and identify possible limitations.

Description of the Experiment

Regression is a supervised learning technique used to predict a continuous numerical output.

The experiment follows: Dataset, Select Features and Target, Preprocess Data, Split Data, Train Regression Model, Generate Predictions, Evaluate Performance and Interpret Results

Students develop a regression model to estimate a continuous health-related variable based on other available attributes.

Detailed Description of Statistical Methods

Linear Regression

A simple linear regression model can be represented as:



Vidyavardhini's College of Engineering and Technology

Department of Artificial Intelligence & Data Science

$$y = \beta_0 + \beta_1 x + \epsilon$$

For multiple predictors:

$$y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \cdots + \beta_p x_p + \epsilon$$

where:

- y = target variable
- x_i = predictor variables
- β_i = model coefficients
- ϵ = error term

Mean Absolute Error

$$MAE = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

It represents the average absolute prediction error.

Mean Squared Error

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

Larger errors are penalized more heavily.

Root Mean Squared Error

$$RMSE = \sqrt{MSE}$$

RMSE is expressed in the same units as the target variable.

Coefficient of Determination



$$R^2 = 1 - \frac{\sum(y_i - \hat{y}_i)^2}{\sum(y_i - \bar{y})^2} R^2$$

indicates the proportion of variation in the target variable explained by the model.

Residual Analysis

A residual is:

$$e_i = y_i - \hat{y}_i$$

Residual analysis helps identify:

- Systematic prediction errors
- Nonlinear patterns
- Unequal variance
- Potential outliers

Implementation:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
```



Vidyavardhini's College of Engineering and Technology

Department of Artificial Intelligence & Data Science

```
url = "https://raw.githubusercontent.com/jbrownlee/Datasets/master/pima-indians-diabetes.data.csv"
```

```
columns = [  
    "Pregnancies",  
    "Glucose",  
    "BloodPressure",  
    "SkinThickness",  
    "Insulin",  
    "BMI",  
    "DiabetesPedigreeFunction",  
    "Age",  
    "Outcome"  
]  
  
df = pd.read_csv(url, names=columns)  
  
df.head()
```

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigreeFunction	Age	Outcome
0	6	148	72	35	0	33.6	0.627	50	1
1	1	85	66	29	0	26.6	0.351	31	0
2	8	183	64	0	0	23.3	0.672	32	1
3	1	89	66	23	94	28.1	0.167	21	0
4	0	137	40	35	168	43.1	2.288	33	1

```
print("Shape of dataset:", df.shape)  
  
print("\nColumn names:")  
print(df.columns)  
  
print("\nDataset information:")  
df.info()
```



Vidyavardhini's College of Engineering and Technology

Department of Artificial Intelligence & Data Science

Shape of dataset: (768, 9)

... Column names:

```
Index(['Pregnancies', 'Glucose', 'BloodPressure', 'SkinThickness', 'Insulin',  
      'BMI', 'DiabetesPedigreeFunction', 'Age', 'Outcome'],  
      dtype='object')
```

Dataset information:

```
<class 'pandas.core.frame.DataFrame'>
```

RangeIndex: 768 entries, 0 to 767

Data columns (total 9 columns):

#	Column	Non-Null Count	Dtype
0	Pregnancies	768 non-null	int64
1	Glucose	768 non-null	int64
2	BloodPressure	768 non-null	int64
3	SkinThickness	768 non-null	int64
4	Insulin	768 non-null	int64
5	BMI	768 non-null	float64
6	DiabetesPedigreeFunction	768 non-null	float64
7	Age	768 non-null	int64
8	Outcome	768 non-null	int64

dtypes: float64(2), int64(7)

memory usage: 54.1 KB

```
▶ X = df.drop("BMI", axis=1)  
y = df["BMI"]
```

```
print("Features:")  
print(X.columns)
```

```
print("\nTarget:")  
print(y.name)
```

... Features:

```
Index(['Pregnancies', 'Glucose', 'BloodPressure', 'SkinThickness', 'Insulin',  
      'DiabetesPedigreeFunction', 'Age', 'Outcome'],  
      dtype='object')
```

Target:

BMI



```
▶ print(df.isnull().sum())
```

```
... Pregnancies      0
     Glucose         0
     BloodPressure   0
     SkinThickness   0
     Insulin         0
     BMI            0
     DiabetesPedigreeFunction  0
     Age            0
     Outcome        0
     dtype: int64
```

```
▶ invalid_columns = [
    "Glucose",
    "BloodPressure",
    "SkinThickness",
    "Insulin",
    "BMI"
]

print("Zero values in important medical attributes:")
print((df[invalid_columns] == 0).sum())
```

```
... Zero values in important medical attributes:
     Glucose      5
     BloodPressure  35
     SkinThickness 227
     Insulin      374
     BMI          11
     dtype: int64
```

```
▶ df[invalid_columns] = df[invalid_columns].replace(0, np.nan)

print(df.isnull().sum())
```

```
... Pregnancies      0
     Glucose         5
     BloodPressure   35
     SkinThickness   227
     Insulin         374
     BMI            11
     DiabetesPedigreeFunction  0
     Age            0
     Outcome        0
     dtype: int64
```



Vidyavardhini's College of Engineering and Technology

Department of Artificial Intelligence & Data Science

```
▶ for column in invalid_columns:
    df[column] = df[column].fillna(df[column].median())

print(df.isnull().sum())
```

```
*** Pregnancies      0
    Glucose          0
    BloodPressure    0
    SkinThickness    0
    Insulin          0
    BMI              0
    DiabetesPedigreeFunction  0
    Age              0
    Outcome          0
    dtype: int64
```

```
▶ X = df.drop("BMI", axis=1)
  y = df["BMI"]

print("X shape:", X.shape)
print("y shape:", y.shape)
```

```
*** X shape: (768, 8)
    y shape: (768,)
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.20,
    random_state=42
)
```

```
print("Training data:", X_train.shape)
print("Testing data:", X_test.shape)
```

```
Training data: (614, 8)
Testing data: (154, 8)
```



Vidyavardhini's College of Engineering and Technology

Department of Artificial Intelligence & Data Science

```
model = LinearRegression()
```

```
model.fit(X_train, y_train)
```

```
...  
▼ LinearRegression ⓘ ⓘ  
LinearRegression()
```

```
y_pred = model.predict(X_test)
```

```
print("Predicted BMI values:")  
print(y_pred[:10])
```

```
Predicted BMI values:  
[30.36372764 33.68942997 31.25007013 32.34808095 31.30777809 30.18350059  
24.62120515 30.96308569 31.13053036 28.6722921 ]
```

```
comparison = pd.DataFrame({  
    "Actual BMI": y_test.values,  
    "Predicted BMI": y_pred  
})
```

```
print(comparison.head(10))
```

```
...  
Actual BMI Predicted BMI  
0          34.0         30.363728  
1          35.7         33.689430  
2          30.8         31.250070  
3          24.6         32.348081  
4          29.9         31.307778  
5          37.7         30.183501  
6          20.4         24.621205  
7          33.8         30.963086  
8          31.3         31.130530  
9          33.7         28.672292
```



Vidyavardhini's College of Engineering and Technology

Department of Artificial Intelligence & Data Science

```
mae = mean_absolute_error(y_test, y_pred)
```

```
print("Mean Absolute Error (MAE):", mae)
```

```
Mean Absolute Error (MAE): 4.475297317675489
```

```
▶ mse = mean_squared_error(y_test, y_pred)
```

```
print("Mean Squared Error (MSE):", mse)
```

```
... Mean Squared Error (MSE): 33.917131715253305
```

```
rmse = np.sqrt(mse)
```

```
print("Root Mean Squared Error (RMSE):", rmse)
```

```
Root Mean Squared Error (RMSE): 5.823841662962113
```

```
r2 = r2_score(y_test, y_pred)
```

```
print("R2 Score:", r2)
```

```
R2 Score: 0.22452960747640804
```

```
▶ print("Regression Model Performance")  
print("-----")  
print("MAE :", mae)  
print("MSE :", mse)  
print("RMSE :", rmse)  
print("R2 :", r2)
```

```
... Regression Model Performance  
-----  
MAE : 4.475297317675489  
MSE : 33.917131715253305  
RMSE : 5.823841662962113  
R2 : 0.22452960747640804
```



▶ `residuals = y_test - y_pred`

```
print("First 10 residuals:")  
print(residuals.head(10))
```

```
... First 10 residuals:  
668    3.636272  
324    2.010570  
624   -0.450070  
690   -7.748081  
473   -1.407778  
204    7.516499  
97    -4.221205  
336    2.836914  
568    0.169470  
148    5.027708  
Name: BMI, dtype: float64
```

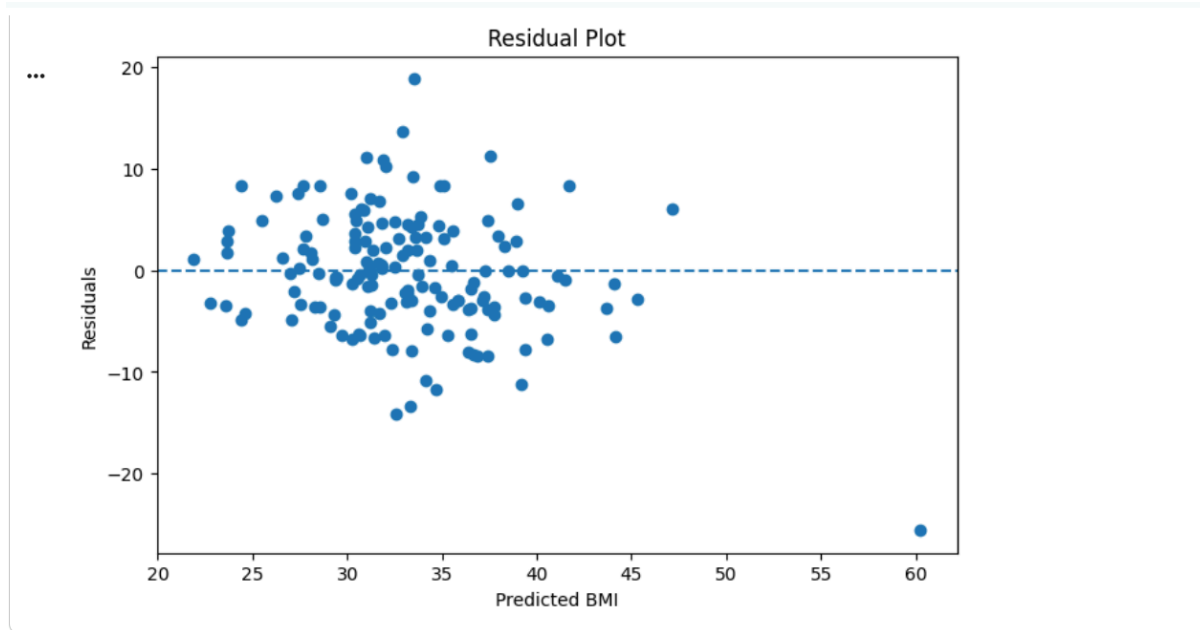
▶ `plt.figure(figsize=(8, 5))`

```
plt.scatter(y_pred, residuals)  
  
plt.axhline(y=0, linestyle="--")  
  
plt.xlabel("Predicted BMI")  
plt.ylabel("Residuals")  
plt.title("Residual Plot")  
  
plt.show()
```



Vidyavardhini's College of Engineering and Technology

Department of Artificial Intelligence & Data Science



```
plt.figure(figsize=(8, 5))

plt.scatter(y_test, y_pred)

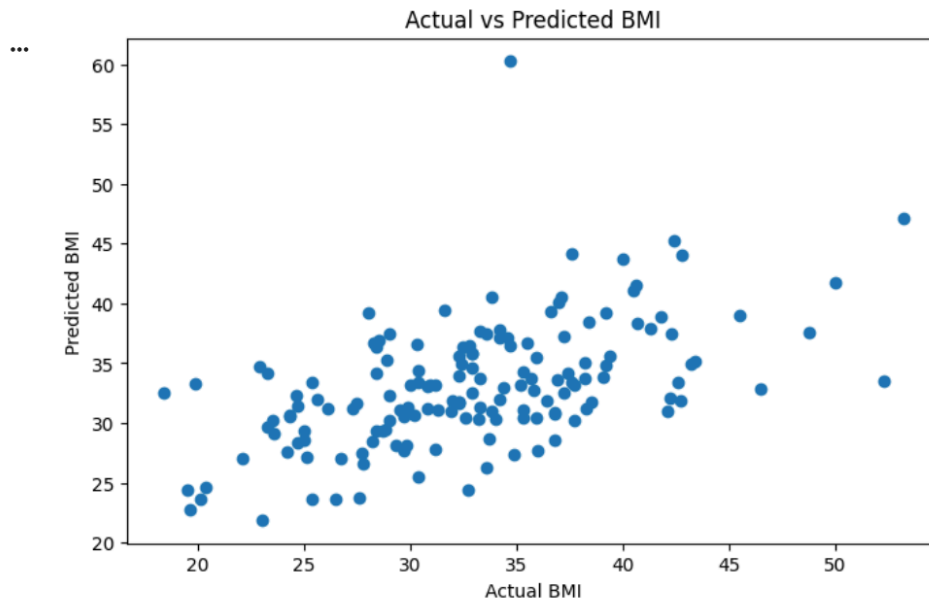
plt.xlabel("Actual BMI")
plt.ylabel("Predicted BMI")
plt.title("Actual vs Predicted BMI")

plt.show()
```



Vidyavardhini's College of Engineering and Technology

Department of Artificial Intelligence & Data Science



```
coefficients = pd.DataFrame({  
    "Feature": X.columns,  
    "Coefficient": model.coef_  
})  
  
print(coefficients)
```

	Feature	Coefficient
0	Pregnancies	-0.037924
1	Glucose	0.010191
2	BloodPressure	0.109209
3	SkinThickness	0.403440
4	Insulin	0.004445
5	DiabetesPedigreeFunction	1.301774
6	Age	-0.101556
7	Outcome	2.824610

```
print("Intercept:", model.intercept_)
```

Intercept: 12.907617645905479

Conclusion

A regression model was developed to predict a continuous health-related variable using the Pima Indians Diabetes Dataset. The model was evaluated using appropriate regression metrics, and residual analysis was performed to understand the model's prediction errors and limitations.



Vidyavardhini's College of Engineering and Technology

Department of Artificial Intelligence & Data Science

Questions to be Answered by Students

1. Which variable was selected as the target, and why was it suitable for regression?

ANS: The BMI (Body Mass Index) variable was selected as the target variable. BMI is a continuous numerical variable and can take different numerical values such as 22.5, 27.8, and 31.4. Therefore, it is suitable for prediction using a regression model.

2. What are the values of MAE, MSE, RMSE, and R^2 obtained by the model?

ANS: Regression Model Performance:

MAE : 4.475297317675489

MSE : 33.917131715253305

RMSE : 5.823841662962113

R^2 : 0.22452960747640804

3. Which performance metric provides the most useful interpretation for this problem? Justify your answer.

ANS: For this experiment, **MAE provides an easy-to-understand interpretation** because it represents the average absolute prediction error in the same units as BMI.

For example, an MAE of 4 means that the model's BMI predictions differ from the actual BMI values by about 4 BMI units on average.

4. What patterns or issues were observed from the residual analysis?

ANS: The residuals were generally scattered around zero, indicating that the regression model captured some of the relationship between the medical attributes and BMI. However, some residuals were relatively large, showing that the model made larger prediction errors for certain observations. No strong systematic pattern was observed, but the presence of these larger residuals suggests that the relationship may not be completely linear and that some observations may act as outliers.



Vidyavardhini's College of Engineering and Technology

Department of Artificial Intelligence & Data Science

5. State **two limitations** of the developed regression model.

1. **ANS: The model assumes a linear relationship** between the predictor variables and BMI, whereas the actual relationships between medical attributes may be nonlinear.
2. **The model may be affected by outliers and data quality issues**, including unusual medical measurements and the replacement of invalid zero values with median values.